A Multi-Agent AI Framework for Stock Market Research and Investment Recommendation

Sujender Kadian[1*]

[1*]Department of Computer Science and Engineering, Bennett University, Greater Noida, Uttar Pradesh, India.

Corresponding author(s). E-mail(s): s24cseu0431@bennett.edu.in;

Abstract

Individual investors often need to examine financial news, company fundamentals, market trends, and investment risks before comparing stocks. This information is distributed across multiple sources and can be time-consuming to synthesize consistently. This paper presents Stock Picker AI, a prototype multi-agent workflow that coordinates specialized language-model agents to identify companies receiving attention in a selected market sector, gather publicly available information, compare candidate companies, and produce a structured investment research report. The workflow is implemented using CrewAI for agent orchestration, a Groq-hosted language model for reasoning, Serper for web search, Python for workflow integration, ReportLab for PDF generation, and SMTP-based email delivery. A manager agent coordinates the trending-company discovery, financial research, and stock-selection stages. The system is designed to make its recommendations more inspectable by separating discovery from research and final decision synthesis, while documenting sources and uncertainty in the generated report. This manuscript describes the system architecture, workflow, implementation choices, and an evaluation plan. Quantitative claims about predictive accuracy or investment returns are not made because a validated historical evaluation dataset and measured results were not supplied. The prototype is intended as a research-support tool, not as a substitute for professional financial advice.

Keywords: multi-agent systems, large language models, financial research, stock analysis, investment decision support, CrewAI, information retrieval

1 Introduction

Investment research requires the synthesis of information from several sources, including financial news, company disclosures, market summaries, and risk-related commentary. Although online information is readily available, investors must still determine which sources are relevant, distinguish facts from opinions, compare companies on consistent criteria, and summarize the evidence behind a potential investment decision. These steps can be difficult to repeat reliably when many companies are under consideration.

Large language models (LLMs) can summarize and organize text, while web-search tools can retrieve recent public information. However, a single unconstrained model call may combine discovery, evidence collection, and recommendation into one opaque response. A multi-agent design offers an alternative: divide the task into specialized stages and make the output of each stage available to the next. The separation does not guarantee correctness, but it can improve workflow structure and make errors easier to inspect.

This paper presents Stock Picker AI, a prototype that automates a research workflow from sector-level company discovery to a final stock-selection report. Its intended outputs include a list of trending companies, a written research report, a decision summary, a PDF document, and an email containing the generated report. CrewAI coordinates the agents, a Groq-hosted model provides language-model reasoning, Serper supports web search, and ReportLab generates the PDF artifact.

The contributions of this work are: (1) a modular multi-agent architecture for stock research; (2) a staged process that separates candidate discovery, financial research, and recommendation synthesis; (3) an automated reporting and email-delivery pipeline; and (4) an evaluation protocol that can be used to assess factuality, evidence coverage, reproducibility, latency, and operational reliability before deployment in a real investment workflow.

2 Related Work

2.1 Financial decision support

Financial decision-support systems combine quantitative indicators, company information, and contextual evidence to help users compare alternatives. Traditional approaches commonly rely on predefined ratios, technical indicators, statistical models, or analyst-authored reports. Such approaches can be interpretable when their inputs and assumptions are explicit, but they may require substantial manual work to gather and summarize unstructured information.

2.2 Large language models for financial text

LLMs can extract themes from text, summarize news, and produce structured explanations. Their outputs can nevertheless contain unsupported claims, outdated information, or incorrect numerical details. In financial applications, generated statements should therefore be checked against source documents and time-stamped data.

A language model's fluent explanation should not be treated as proof that a claim is true.

2.3 Multi-agent workflows

Multi-agent LLM systems assign different subtasks to specialized agents and coordinate their outputs through a workflow or manager. This can encourage task separation and make intermediate artifacts available for review. However, agents may share the same model limitations, propagate errors from earlier stages, or produce inconsistent conclusions. The design in this paper uses staged outputs and recommends explicit evidence checks rather than assuming that agent collaboration alone improves accuracy.

3 System Overview

Stock Picker AI is organized as a pipeline of coordinated tasks. The user supplies a market sector or other search scope. A manager agent initiates the workflow and delegates company discovery to a trending-company agent. The resulting candidate list is passed to a financial research agent, which gathers available public information and prepares comparable research notes. A stock-picker agent then synthesizes the evidence and produces a decision summary. Finally, the application generates a PDF report and sends it through an email workflow when the required configuration is available.

3.1 Manager agent

The manager coordinates the order of operations, passes outputs between tasks, and checks whether the expected artifacts have been produced. It is responsible for workflow coordination rather than independently serving as the sole source of financial evidence.

3.2 Trending company finder

The trending-company finder searches for companies receiving attention within the user's selected scope. Search results are treated as candidate-discovery signals, not as proof that a company is financially sound or suitable for investment. The stage should preserve company names, the reason each candidate was surfaced, source links, and retrieval dates wherever available.

3.3 Financial researcher

The financial researcher collects and organizes information about the candidate companies. Depending on source availability, this can include company descriptions, reported financial information, business developments, sector context, and risks mentioned in public sources. The prototype uses web search as a retrieval mechanism; it should not be assumed to have direct, complete, or real-time access to every exchange feed or company filing. Numeric values should be associated with their source, reporting period, currency, and retrieval timestamp.

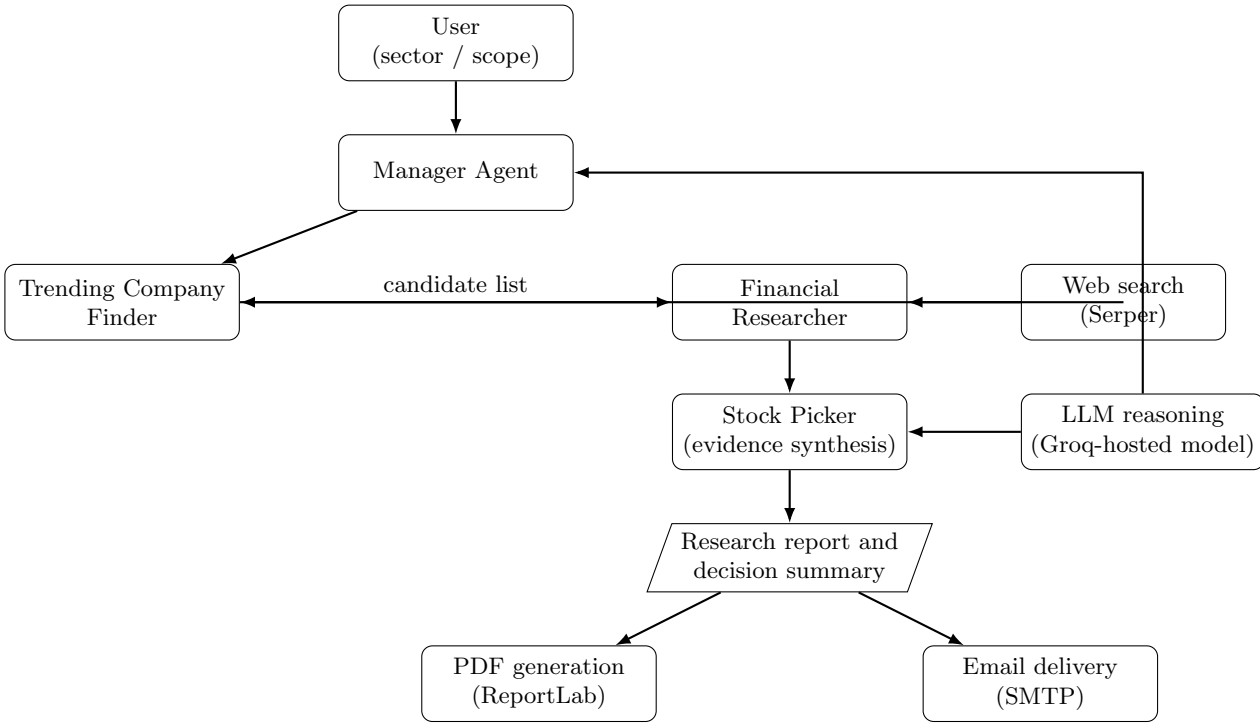


Fig. 1 Conceptual architecture of Stock Picker AI. The manager coordinates discovery and research agents; search and language-model services support the relevant stages, and the final artifacts are rendered as a PDF and delivered by email.

3.4 Stock picker

The stock picker compares the research notes and produces a recommendation-oriented summary. A recommendation should be supported by cited evidence, explicit assumptions, and a discussion of risks and missing information. If the evidence is insufficient or conflicting, the system should state that limitation rather than force a confident selection. The output is decision support and is not personalized financial advice.

4 Methodology

4.1 Input and task orchestration

The main input is a user-defined sector or search scope. The orchestrator starts the company-discovery task, stores its output, and supplies the candidate list to the research stage. The research stage creates a structured summary for each candidate. The stock-picker stage receives these summaries and produces the final decision document. Each stage should have a defined output format so that missing fields and malformed responses can be detected before downstream processing.

4.2 Evidence collection and source handling

Web search provides candidate URLs and text snippets that guide further research. Search snippets alone may be incomplete or stale; important claims should be checked against the underlying page or a primary source where possible. The implementation should retain URLs and retrieval timestamps and should distinguish reported facts from model-generated interpretation. If a source does not provide a value, the system should record it as unavailable rather than infer a number.

4.3 Recommendation synthesis

The prototype's decision stage synthesizes the collected evidence rather than relying on a validated predictive model. For a more reproducible comparison, candidate stocks can be assessed against a fixed rubric, for example:

$$S_i = \sum_{k=1}^{K} w_k r_{ik}, \qquad \sum_{k=1}^{K} w_k = 1, \tag{1}$$

where S_i is the comparison score for company i, r_{ik} is its normalized assessment on criterion k, and w_k is the predefined weight for that criterion. This equation describes a possible extension to the prototype; it does not imply that calibrated weights or a validated scoring model have already been implemented. Criteria and weights should be documented before evaluation, and missing values should not silently be treated as favorable scores.

4.4 Report generation and delivery

The pipeline creates intermediate artifacts such as the candidate list, research notes, and decision summary. ReportLab is used to render the final content as a PDF. SMTP can be used to deliver the report by email when valid credentials and recipient configuration are supplied. Secrets such as API keys and mail passwords should be stored in environment variables or a secrets manager, not committed to source control. Delivery errors should be logged without exposing credentials.

5 Implementation

The prototype is implemented in Python. CrewAI provides agent and task orchestration, and a Groq-hosted language model is used for language-model reasoning. Serper provides web-search functionality. ReportLab generates the PDF report, while SMTP supports email delivery. Pydantic can be used to validate structured outputs and reject responses that do not conform to the expected schema. The project repository has been referred to as `Stock-Picker-Agent` in the development workflow.

The exact availability and quality of financial information depend on the pages returned by search and the services configured at runtime. The system should therefore expose its sources and retrieval times and should not describe search-based retrieval as

Algorithm 1 Stock Picker AI workflow

Require: User-defined sector or search scope q
Ensure: Research artifacts and, when configured, a PDF email

Initialize the workflow manager and required service clients
Retrieve candidate companies relevant to q
Preserve candidate names, source references, and retrieval times
for all candidate companies c **do**
    Retrieve available public information about c
    Build a structured research summary with evidence and caveats
end for
Compare candidates using the available evidence and stated criteria
Generate a decision summary, including risks and missing information
Render the research artifacts into a PDF report
if email configuration is valid **then**
    Send the report using the configured email service
else
    Save the report and indicate that email delivery was not configured
end if
return the generated artifacts and execution status

Table 1 Main implementation components of the prototype.

Component	Technology	Role
Agent orchestration	CrewAI	Coordinates specialized agents and task dependencies
Language-model inference	Groq-hosted model (configured as `openai/gpt-oss-20b` in the project context)	Generates research summaries and synthesizes evidence
Web retrieval	Serper API	Discovers public web pages and recent information
Application logic	Python	Connects agents, tools, validation, and file operations
Output validation	Pydantic	Supports schema validation for structured agent outputs
PDF reporting	ReportLab	Converts research artifacts into a PDF report
Email delivery	SMTP	Sends the generated report when configured

a guaranteed market-data feed. Model names and API behavior can change; deployment documentation should record the exact model identifier and package versions used for each experiment.

Table 2 Proposed evaluation protocol; results must be populated after experiments.

Dimension	Measurement	Current status
Claim support	Audited supported claims / all audited claims	Not measured
Research coverage	Required evidence fields populated / total required fields	Not measured
Workflow reliability	Successful runs / total runs	Not measured
Execution latency	Median and 95th-percentile runtime	Not measured
Email delivery	Successfully delivered reports / send attempts	Not measured
Investment performance	Out-of-sample performance with costs and baseline comparison	Not measured

6 Evaluation Plan

No benchmark results, validated historical dataset, or measured performance figures were supplied for this manuscript. Accordingly, this section defines an evaluation protocol rather than claiming that the prototype has achieved a particular accuracy, return, or risk-adjusted performance.

6.1 Research questions

The evaluation should answer four questions: (1) Are factual statements in the generated report supported by the cited sources? (2) Does the workflow cover the required research dimensions consistently across candidate companies? (3) Can repeated runs reproduce the same source-backed conclusions when inputs and source snapshots are held constant? (4) What are the end-to-end latency, service-failure rate, and report-delivery success rate?

6.2 Suggested metrics

Factual support can be measured by manually auditing a sample of report claims and calculating the proportion supported by their cited sources. Coverage can be measured as the proportion of required fields populated with verifiable evidence. Reliability can be measured by successful workflow completions divided by total runs. Latency should be reported using median and 95th-percentile end-to-end execution time. Email delivery success should be recorded separately from research quality. If the system is later evaluated as a return-predicting strategy, the evaluation must use point-in-time data, include transaction costs and realistic execution assumptions, avoid look-ahead bias, and compare against appropriate baselines.

7 Discussion

The main value of Stock Picker AI is the organization of a multi-step research task into explicit stages and reusable artifacts. Separating candidate discovery from financial research reduces the risk that a trending company is automatically treated as a good

investment. Producing a report also gives a human reviewer a tangible artifact to inspect before acting on the output.

The system has important limitations. First, web search may return incomplete, promotional, outdated, or secondary information. Second, LLM-generated summaries may contain factual errors or omit material caveats. Third, the prototype does not establish that its recommendations outperform a market index or any alternative strategy. Fourth, trending-company discovery can introduce popularity bias and may overemphasize short-term news. Finally, market suitability depends on an investor's financial position, goals, time horizon, and risk tolerance, none of which can be inferred reliably from a sector query alone.

These limitations motivate several safeguards: prefer primary sources for material claims; attach source links and timestamps; validate structured outputs; flag missing or contradictory evidence; log failures; and require human review before any investment action. API keys and email credentials must be protected, and the report should clearly communicate that it is informational decision support.

8 Conclusion and Future Work

This paper described Stock Picker AI, a multi-agent prototype for discovering candidate companies, gathering public financial information, synthesizing a research-based recommendation, generating a PDF report, and delivering the report by email. The architecture uses CrewAI, a Groq-hosted language model, Serper search, Python, ReportLab, and SMTP. The paper also provided a workflow algorithm, a system architecture diagram, and a proposed evaluation protocol.

The next steps are to implement claim-level source tracking, validate agent outputs with explicit schemas, add a reproducible scoring rubric, and evaluate the workflow on a fixed set of companies and time-stamped source snapshots. Any claim about recommendation quality or investment performance should be made only after a documented out-of-sample study. The system should remain a human-reviewed research assistant rather than an autonomous financial adviser.

Declarations

- **Funding:** No external funding was specified for this work.
- **Conflict of interest:** The author declares no competing interests relevant to this manuscript.
- **Ethics approval and consent to participate:** Not applicable. The work describes a software prototype and does not report experiments involving human participants or animals.
- **Consent for publication:** Not applicable.
- **Data availability:** The prototype relies on publicly accessible web information retrieved at runtime. No curated evaluation dataset was supplied with this manuscript.
- **Materials availability:** Not applicable.

- **Code availability:** The project has been referred to as `Stock-Picker-Agent`; a public repository URL was not provided in the information available for this manuscript.
- **Author contribution:** Rishi Ranjan developed the project concept and prototype and prepared this manuscript.

Acknowledgements

The author acknowledges the open-source software and API tooling used in the prototype, including CrewAI, Python, ReportLab, and the configured search and model services.

References

[1] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I (2017) Attention is all you need. In: Advances in Neural Information Processing Systems 30.

[2] Brown TB, Mann B, Ryder N, et al (2020) Language models are few-shot learners. In: Advances in Neural Information Processing Systems 33:1877–1901.

[3] CrewAI (n.d.) CrewAI documentation. Available at: https://docs.crewai.com/. Accessed 30 October 2026.

[4] Groq (n.d.) Groq documentation. Available at: https://console.groq.com/docs/. Accessed 30 October 2026.

[5] Serper (n.d.) Serper API documentation. Available at: https://serper.dev/. Accessed 30 October 2026.

[6] ReportLab (n.d.) ReportLab documentation. Available at: https://www.reportlab.com/docs/. Accessed 30 October 2026.

[7] Pydantic (n.d.) Pydantic documentation. Available at: https://docs.pydantic.dev/. Accessed 30 October 2026.